

Livello 4 – Livello trasporto

Collega livello rete con livello sessione iso/osi o livello applicazione tcp/ip.

Fornisce un collegamento efficace tra host origine e host destinazione, indipendentemente dalla rete usata.

Svolge funzioni simili al livello 2 (controllo errori, sequenza corretta dati, controllo di flusso), trattando una connessione logica anziché fisica come al livello 2

2 tipi di **servizi**:

- **TCP** (transmission control protocol): connection oriented
- **UDP** (user datagram protocol): connectionless.

Il formato di un indirizzo di livello 4 è scritto nel formato ip:numPorta dove il numero di porta è un intero a 16 bit. ip:porta si chiama socket.

La socket è contenuta negli header di livello 3 e 4 e identifica in modo univoco un processo applicativo (client o server) in esecuzione su un host.

Svolge funzioni di multiplexing / demultiplexing

- Multiplexing: in trasmissione il livello trasporto riceve i dati dalle socket e aggiunge le proprie informazioni di controllo (header)
- Demultiplexing: in ricezione il livello trasporto legge l'header e determina a quale socket consegnare i dati

Tipi di porte:

- Well known: applicazioni più importanti, con numero di porta fisso (http, ftp, dns...)
- Dinamiche (temporanee): numeri dinamici tra 0 e 65535

Primitive del livello trasporto: metodi di comunicazione

- LISTEN: attesa di connessione
- SEND DATA: invio di un contenuto
- RECEIVE DATA: ricezione di un contenuto
- T-CONNECT: apertura della connessione
- T-DISCONNECT: chiusura della connessione

Metodi definiti per ogni primitiva:

- request(): chiedo al servizio di fare qualcosa

- indication(): si viene avvertiti dal servizio di un evento
- response(): chiedo al servizio di rispondere a un evento
- confirm(): il servizio segnala l'arrivo di una conferma

Esempi:

- Connessione asincrona (nessuna conferma al mittente – connectionless):
`primitiva.request();` `primitiva.indication()`
- Connessione sincrona (con conferma – connection oriented):
`primitiva.request();` `primitiva.indication()`
`primitiva.response();` `primitiva.confirm()`

Per stabilire una connessione sincrona tra A e B:

ENTITÀ	AZIONE	DESCRIZIONE
A	invia <code>T-CONNECT.request()</code>	A vuole connettersi
B	riceve <code>T-CONNECT.indication()</code>	qualcuno vuole connettersi a B
B	invia <code>T-CONNECT.response()</code>	B accetta la connessione
A	riceve <code>T-CONNECT.confirm()</code>	conferma di connessione stabilita

Differenze e rispettivi vantaggi tra **UDP** e **TCP**:

UDP:

- Ogni datagram UDP è indipendente, è un servizio best effort senza garanzia di consegna dati.
- È simile a IP, ma ha capacità di distinguere fra più destinazioni (porte) all'interno dello stesso host. I datagram non arrivati non vengono ritrasmessi, e se arrivano in disordine non vengono riordinati (se non dal S.O.).
- Ha un header (intestazione) a 8 byte. Supporta il multicast. È incapsulato dentro il pacchetto IP (come tcp), è formato da header e payload e non può superare la dimensione massima della parte dati del datagramma IP.
- Non contiene indirizzi (ip) di mittente e destinatario!

Vantaggi UDP

- **Non richiede di stabilire una connessione:** non introduce un ritardo dovuto alla fase di setup della connessione
- **Non mantiene lo stato della connessione:** un server può supportare molti più client attivi
- Il **sovraccarico** dovuto all'intestazione del pacchetto è **minimo**: solo 8 byte contro i 20 di un protocollo connection-oriented
- Il **controllo del livello applicativo è più efficace**: il mittente non viene mai bloccato

TCP:

- più diffuso, affidabile, riordina i pacchetti in ricezione.
- È possibile costruire applicazioni che si basano sul trasferimento di file senza errori tra host remoti.
- Effettua il controllo di congestione end-to-end (condivisione equa delle risorse).
- Ma: richiede molte più risorse rispetto a UDP, ovviamente.
- Sfrutta connessioni solo di tipo full-duplex.
- È adottato in combinazione con protocolli di livello 3, ad esempio IP.
- Suddivide i dati in segmenti di massimo 64KB e li incapsula in datagrammi IP, come udp
- Ha intestazione/header a 20 byte
- I pacchetti devono rispettare i limiti imposti dalle dimensioni dei pacchetti delle reti che devono attraversare (MTU – Max. Transmission Unit)
- Campi tcp:
 - o Sequence number, 32 bit: numero di sequenza progressivo del primo byte di dati contenuto nel segmento
 - o Checksum, 16 bit: usato per la verifica di validità del segmento. In udp è facoltativo, in tcp obbligatorio
 - o Acknowledgment number, 32 bit: numero di riscontro che conferma la ricezione di un pacchetto
 - o Data offset, 4 bit: indica la lunghezza dell'header nel segmento TCP. È diverso dal campo Data, che è un'altra cosa.

La connessione TCP:

Apertura:

- Handshaking fra client e server: stabilisce un canale dedicato
 - Three-way handshake: 3 messaggi tra mittente e destinatario per concordare i parametri di trasmissione
1. Il client genera il pacchetto con il quale vuole aprire la connessione. Particolarizzato da sequence number, ack, fin e syn. Il bit synchronize è a 1 in questo caso, gli altri invece sono a 0.
 2. Risposta del server con un pacchetto che ha sequence number aumentato di 1 rispetto a quello ricevuto, oltre syn, in questo pacchetto anche ack è a 1 (infatti è un pacchetto di ack)

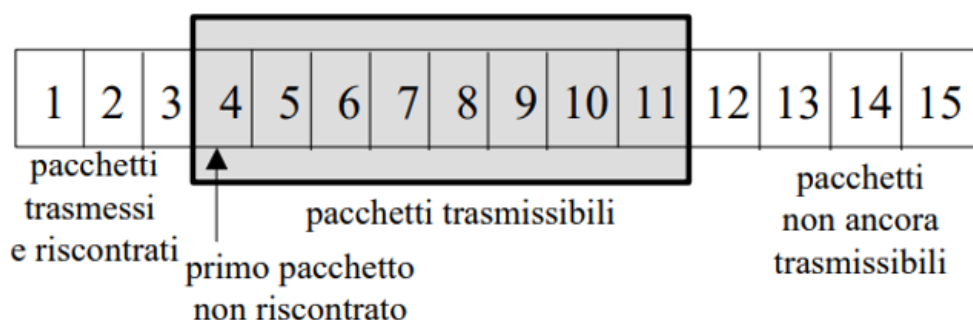
3. Il mittente manda un messaggio con cui apre la connessione, dove il sequence number è uguale all'ack number e il bit di fin va a 1.

Chiusura:

- Deve avvenire in entrambi i sensi (non è unica, ma va fatta sia per mittente che per destinatario)
- Handshake a 3 vie: connessioni chiuse contemporaneamente, simile a quello di apertura ma il flag usato qui è fin invece che syn.
- Handshake a 4 vie: connessioni chiuse una per volta. Uno dei due terminali invia la richiesta di FIN e attende l'ack, l'altro fa lo stesso, portando alla generazione di 4 pacchetti appunto.
- Il server poi invia l'ack e fa la chiusura

TCP – Ritrasmissione

- Quando il trasmettitore deve inviare un pacchetto, nel momento in cui lo spedisce ne mantiene una copia e fa partire un timer. Se allo scadere del timer non ha ancora ricevuto il relativo ACK ritrasmette il pacchetto.
- Per evitare problemi di duplicazione, i pacchetti vengono numerati, e l'ack contiene il numero del pacchetto di cui vuole fornire riscontro
- L'attesa di ack rallenta la trasmissione, quindi si utilizzano tecniche per alleviare il problema, ad esempio Sliding window
- Un pacchetto si dice non riscontrato quando non è ancora stato ricevuto il suo riscontro
- La finestra del trasmettitore inizia col primo pacchetto non riscontrato. I pacchetti successivi, che stanno dentro alla finestra, possono essere trasmessi, mentre i pacchetti che seguono la finestra non possono essere trasmessi.
- Quando un pacchetto P (trasmesso) nella finestra viene riscontrato (e sono riscontrati anche i suoi precedenti) allora la finestra avanza fino al pacchetto successivo a P, consentendo di trasmettere i pacchetti che sono entrati nella finestra.



- La dimensione delle finestre nella sliding window influenza l'efficacia della trasmissione: molto grosse fanno diminuire il periodo di non utilizzo della rete ma aumentano la gravità di eventuali errori, ma scegliendo una dimensione opportuna si può mantenere la rete satura di pacchetti senza congestionarla
- Il **vantaggio** è che questi riscontri cumulativi sono facili da generare, e che la perdita di un riscontro non costringe alla ritrasmissione se arriva un riscontro per un byte successivo.
- Uno **svantaggio** è che il trasmettitore, se un riscontro per un byte non arriva, non è in grado di capire se manca solo un pezzetto (un buco anche di quel solo byte) o manca tutto da quel byte in avanti, e ricomincia a trasmettere tutto da quel byte.